AI Powered Pothole Detection and Intelligent Route Optimization System

Zain Mahedi (1WA23CS056) Thanu George (1WA23CS019)
Vikas Shashi (1WA23CS043) Tanish Agrawal (1WA23CS008)
Department of Computer Science and Engineering

Under the guidance of
Dr. Seema Patil
Associate Professor, Department of CSE, BMSCE

Introduction

Potholes damage vehicles, slow traffic and cause accidents. India recorded 4,446 pothole-related road accidents and 1,856 deaths in 2022 (MoRTH). Most potholes are still found through citizen complaints and manual surveys, which are slow and rarely record the exact location, while navigation apps choose routes by distance and traffic alone.

This project detects potholes in road video, places them on an OpenStreetMap road graph, and recommends routes that trade a little extra distance for fewer expected potholes. It is a ground-up rebuild of our earlier segmentation-based prototype, with leakage-controlled data, a measured baseline and an evaluation protocol that reports uncertainty.

Objectives

1. Prepare RDD2022 and HRP4K as one pothole class with near-duplicate grouping and leakage-free splits.
2. Train a reproducible YOLOv8s detection baseline and archive it with its configuration and hash.
3. Evaluate with thresholds fixed on validation data, group-bootstrap 95% intervals and paired comparisons.
4. Fine-tune on forward-view HRP4K footage at 1280 px and compare against the baseline on held-out test sets.
5. Build the detection-to-map pipeline: video and GPS in, tracked, geotagged and road-matched potholes out.
6. Recommend pothole-aware routes through a tested FastAPI backend, SQLite store and web interface.

Proposed System

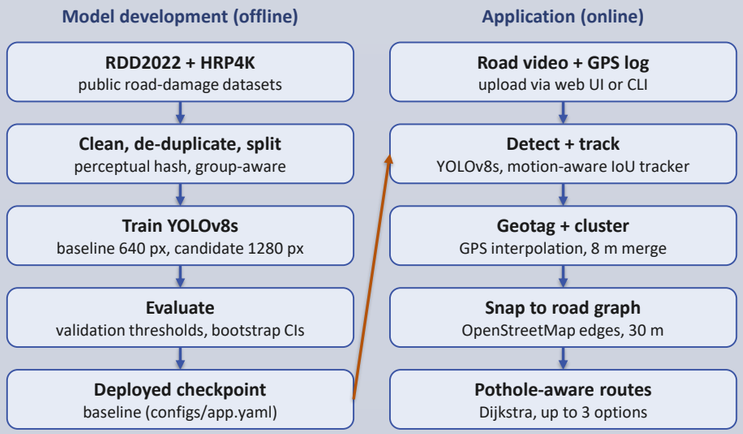


Fig 1. System overview. Orange arrow: the deployed checkpoint drives the application's detector.

Road condition is summarised per road segment as potholes per 100 m of surveyed road, and each recommended route reports how much of it was actually surveyed (adequate, partial or insufficient evidence).

Methodology

1) Dataset preparation

RDD2022 (pothole class D40) and HRP4K (4K forward-view frames) were merged into one pothole class. Near-duplicates were grouped by perceptual hash (Hamming distance ≤ 8) and each group kept in one split. Review only excluded images; no box was added or moved. A 203-image manhole set is evaluation-only.

Source	Train	Validation	Test
RDD2022 (images / boxes)	8,220 / 5,017	2,784 / 477	2,784 / 503
HRP4K (images / boxes)	3,973 / 4,709	825 / 922	811 / 836
Total	12,193 / 9,726	3,609 / 1,399	3,595 / 1,339

Table 1. Final dataset after cleaning and leakage-controlled splitting.



Fig 2. RDD2022 examples with pothole boxes (India, Japan). RDD2022, CC BY-SA 4.0.

2) Model training

	Baseline (deployed)	Candidate (experimental)
Initialisation	YOLOv8s, COCO weights	Baseline checkpoint
Training data	RDD2022	RDD2022 + HRP4K
Input size	640 px	1280 px
Optimiser	MuSGD, lr 0.01	AdamW, lr 0.001, cosine
Epochs	76 of 100 epochs run	50; checkpoint picked on val.
Hardware	RTX 5090 (cloud)	RTX 5090, 79 min, about $0.77

3) Evaluation protocol

- Threshold fixed at max F1 on validation, then applied unchanged to held-out test.
- 95% intervals by group bootstrap (1,000 resamples); differences paired on identical resamples.
- Per dataset: mAP50, mAP50-95, precision, recall, F1, false positives per image, manhole hits.

4) Detection-to-map pipeline and routing

Detections are linked across frames by a motion-aware IoU tracker, geotagged by interpolating the GPS log, merged into mapped potholes within 8 m and snapped to the nearest OpenStreetMap edge (2,996 nodes, 7,138 edges in Bengaluru). Dijkstra's algorithm returns up to three routes, each edge costed as:

$$cost = L + a \times 300\ m \times E, \qquad E = n + r\,(L - s) / 100$$

L length, a avoidance setting, n potholes seen, s surveyed length, r assumed rate on unsurveyed road

Results

25× higher mAP50 on forward-view HRP4K test (0.018 → 0.454)

3,107 → 0 whole-frame false boxes vs our earlier system

17 ms per 1280 px frame on a laptop RTX 4060

1,101 automated tests passing

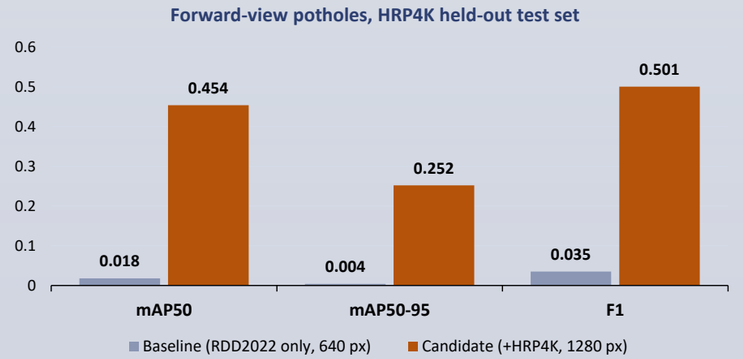


Fig 3. Fine-tuning on forward-view footage lifts every metric (95% intervals exclude zero).

Held-out test set (mAP50)	Baseline 640	Candidate 1280	Difference [95% CI]
HRP4K (forward view)	0.018	0.454	+0.435 [0.392, 0.479]
Manhole probe set	0.368	0.427	+0.058 [−0.027, 0.195]
RDD2022	0.497	0.453	−0.045 [−0.086, −0.006]

Table 2. Paired comparison; thresholds chosen on validation, applied unchanged to test.

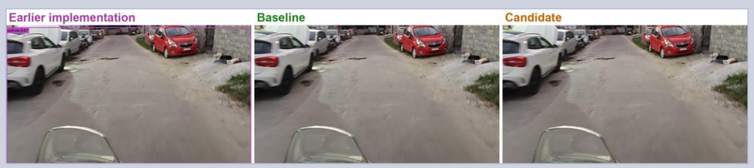


Fig 4. Our own Bengaluru drive: the earlier system boxes the whole street as a pothole (0.87); both new models correctly report nothing. Qualitative; no ground truth.

Conclusion

We rebuilt the system end to end: leakage-controlled data, a reproducible YOLOv8s baseline (mAP50 0.497 on held-out RDD2022), a detection-to-map pipeline and pothole-aware routing behind a tested API. Fine-tuning at 1280 px makes the detector work on forward-view dashcam footage (mAP50 0.018 → 0.454) while still running in real time on a laptop GPU, at a training cost of about $0.77. Because it gives up some recall on RDD2022, the baseline stays deployed until labelled frames from our own roads decide between them.